

Numerical Solution of the Heston PDE for the European Call using Parallel Cyclic Reduction

Vincent Candela

Problem Statement

Asset Price and Variance SDEs

$$\begin{aligned}dS_t &= \mu S_t dt + \sqrt{v_t} S_t dW_t^S \\ dv_t &= \kappa(\nu - v_t) dt + \sigma \sqrt{v_t} dW_t^v\end{aligned}$$

Heston PDE

$$\frac{\partial u}{\partial t} = \frac{1}{2} v s^2 u_{ss} + r s u_s + \frac{1}{2} \sigma^2 v u_{vv} + \kappa(\nu - v) u_v + \rho \sigma v s u_{sv} - r u$$

- $u = u(s, v, t)$: option price as a function of asset price (s), variance (v), and time (t)
- s : underlying asset price (spot)
- v : instantaneous asset variance
- r : risk-free interest rate
- ν : long-run variance
- κ : mean-reversion speed of v toward ν
- σ : volatility of the variance process (vol of vol)
- ρ : correlation between Wiener Processes W^S and W^v

Crank-Nicolson

- Trapezoid Rule in Time $\frac{u^{n+1} - u^n}{dt} = \frac{1}{2}(Lu^{n+1} + Lu^n)$
- Crank-Nicolson $\left(I - \frac{dt}{2}L\right)u^{n+1} = \left(I + \frac{dt}{2}L\right)u^n$

Alternating Direction Implicit (ADI)

- 2D Courant-Friedrichs-Lewy (CFL) Condition

$$\Delta t \left(\frac{D_s}{(\Delta s)^2} + \frac{D_v}{(\Delta v)^2} \right) \leq \frac{1}{2}$$
- Update Right Hand Side

$$u_{i,j}^{rhs} = u_{i,j} + \frac{1}{2} dt \left(\frac{1}{2} v s^2 u_{ss} + r s u_s + \frac{1}{2} \sigma^2 v u_{vv} + \kappa (\nu - v) u_v + \rho \sigma v s u_{sv} - r u_{i,j} \right)$$
- S Correction:
Partial Update

$$\left[I - \frac{1}{2} dt \left(\frac{1}{2} v s^2 u_{ss} + r s u_s - \frac{1}{2} r \right) \right] u_{i,j}^{intermediate} = rhs_{i,j}$$
- V Correction:
Final Update

$$\left[I - \frac{1}{2} dt \left(\frac{1}{2} \sigma^2 v u_{vv} + \kappa (\nu - v) u_v - \frac{1}{2} r \right) \right] u_{i,j}^{n+1} = u_{i,j}^{intermediate}$$

Boundary Condition

- Dirichlet in s
- Neumann zero in v

$$u_{rhs}^n_{i,j} = \begin{cases} 0 & i = 0 \quad (\text{out of the money}) \\ \max(s_{\max} - Ke^{-r\tau}, 0) & i = n_s - 1 \quad (\text{discounted } u \text{ at the strike}) \\ u_{i,1}^n & j = 0 \quad (v \text{ at } v_{\min}) \\ u_{i,n_v-2}^n & j = n_v - 1 \quad (v \text{ at } v_{\max}) \end{cases}$$

Parallel Cyclic Reduction

Step 0;

Calculate coefficients a, b, c from the Crank-Nicolson / ADI conditions

Step 1:

$$a_k u_{k-1} + u_k + c_k u_{k+1} = d_k$$

Step 2:

$$u_{k-2}(-a_k a_{k-1}) + u_k(1 - a_k c_{k-1} - c_k a_{k+1}) + u_{k+2}(-c_k c_{k+1}) = -a_k d_{k-1} - c_k d_{k+1} + d_k$$

Step 3:

- $a_k = (-a_k a_{k-1})$
- $b_k = (1 - a_k c_{k-1} - c_k a_{k+1})$
- $c_k = (-c_k c_{k+1})$
- $d_k = -a_k d_{k-1} - c_k d_{k+1} + d_k$

Step 4:

Repeat which results in step size doubling

Finish when step size is larger than grid dimension

Accesses that are out of bounds are coefficients of zero (little movement at boundaries), which allows for pass-through

Parallelization: main()

- NX: s gridpoints
- NY: v gridpoints
- tri_solve_s:
 - NY independent systems
- tri_solve_v:
 - NX independent systems

```
for(int step = 0; step < NITER; step++) {  
    // combined kernel  
    float tau = (step + 1) * DT;  
    rhs_combined<<<NY,NX>>>(d_u_old, d_rhs, DT, ds, dv,  
    RHO, SIGMA, R, LAMBDA, XI, NU, KAPPA, K, tau ,SMIN, VMIN);  
  
    // implicit  
    tri_solve_s<<<NY,NX>>>(d_rhs, d_u_half,  
    DT, ds, dv, R, LAMBDA, XI,  
    SMIN, VMIN  
    );  
    tri_solve_v<<<NX,NY>>>(d_u_half, d_u_new,  
    DT, ds, dv,  
    SIGMA, R, LAMBDA, KAPPA, NU,  
    SMIN, VMIN  
    );  
  
    float *temp = d_u_old;  
    d_u_old = d_u_new;  
    d_u_new = temp;  
}
```

tri_solve_v() kernel: Double Lock:

```
for(int step = 1; step < NY; step *= 2){  
    float a_left = 0.0f;  
    float c_left = 0.0f;  
    float d_left = 0.0f;  
    float a_right = 0.0f;  
    float c_right = 0.0f;  
    float d_right = 0.0f;  
  
    if(j - step >= 0){  
        a_left = a[j - step];  
        c_left = c[j - step];  
        d_left = d[j - step];  
    }  
    if(j + step < NY){  
        a_right = a[j + step];  
        c_right = c[j + step];  
        d_right = d[j + step];  
    }  
    float bb_new = 1.0f - aa * c_left - cc * a_right;  
    float aa_new = -aa * a_left;  
    float cc_new = -cc * c_right;  
    float dd_new = dd - aa * d_left - cc * d_right;  
    // diagonal normalization  
    float inverse = 1.0f / bb_new;  
  
    aa = aa_new * inverse;  
    cc = cc_new * inverse;  
    dd = dd_new * inverse;  
  
    __syncthreads();  
    a[j] = aa;  
    c[j] = cc;  
    d[j] = dd;  
    __syncthreads();  
}
```


tri_solve_v() kernel: Single Lock:

```
for(int step = 1; step < NY; step *= 2){
    float a_left = 0.0f;
    float c_left = 0.0f;
    float d_left = 0.0f;
    float a_right = 0.0f;
    float c_right = 0.0f;
    float d_right = 0.0f;

    if(j - step >= 0){
        a_left = a_in[j - step];
        c_left = c_in[j - step];
        d_left = d_in[j - step];
    }
    if(j + step < NY){
        a_right = a_in[j + step];
        c_right = c_in[j + step];
        d_right = d_in[j + step];
    }

    float bb_new = 1.0f - aa * c_left - cc * a_right;
    float aa_new = -aa * a_left;
    float cc_new = -cc * c_right;
    float dd_new = dd - aa * d_left - cc * d_right;
    // diagonal normalization
    float inverse = 1.0f / bb_new;

    aa = aa_new * inverse;
    cc = cc_new * inverse;
    dd = dd_new * inverse;

    a_out[j] = aa;
    c_out[j] = cc;
    d_out[j] = dd;
    __syncthreads();

    float *temp;
    temp = a_out; a_out = a_in; a_in = temp;
    temp = c_out; c_out = c_in; c_in = temp;
    temp = d_out; d_out = d_in; d_in = temp;
}
```

__shared__ Memory Accesses

- tri_solve_s

```
int i = threadIdx.x;  
int j = blockIdx.x;  
__shared__ float a[NX];  
__shared__ float c[NX];  
__shared__ float d[NX];
```

```
for(int step = 1; step < NY; step *= 2){  
    __syncthreads();  
    a[i] = aa;  
    c[i] = cc;  
    d[i] = dd;  
    __syncthreads();  
}
```

- tri_solve_v

```
int j = threadIdx.x;  
int i = blockIdx.x;  
__shared__ float a[NY];  
__shared__ float c[NY];  
__shared__ float d[NY];
```

```
for(int step = 1; step < NX; step *= 2){  
    __syncthreads();  
    a[j] = aa;  
    c[j] = cc;  
    d[j] = dd;  
    __syncthreads();  
}
```

Limits to P100 Single-GPU Parallelism

- Double `__syncthreads()`: $3 * 4B * \text{Length} = (24 * \text{Length})$ Bytes shared mem
 - Failure at sidelength = 8192
- Single `__syncthreads()`: $6 * 4 B * \text{Length} = (12 * \text{Length})$ Bytes shared mem
 - Failure at sidelength = 4096

Arithmetic Intensity

$$AI = \frac{\text{FLOPs}}{\text{Bytes Read}}$$

- rhs_combined

$$AI = \frac{55}{4 * 9} = 1.53$$

- tri_solve_s

$$AI = \frac{26 + 14 \log_2 NY}{4 * (1 + 6 \log_2 NY)}$$

- tri_solve_v

$$AI = \frac{27 + 14 \log_2 NY}{4 * (1 + 6 \log_2 NY)}$$

Passing Parameters vs Macros

```
__global__ void tri_solve_s(  
    // int nx,  
    // int ny,  
    const float * __restrict__ rhs,  
    float * __restrict__ u_new,  
    float dt,  
    float ds,  
    float dv,  
    // float sigma,  
    float r,  
    float lambda,  
    float xi,  
    float s_min,  
    float v_min  
) {
```

Results

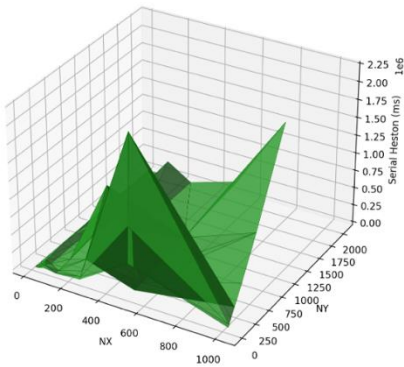


Figure 1: Serial Heston (ms)

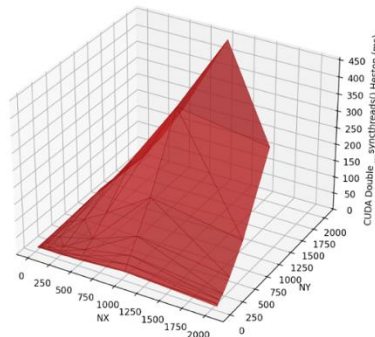


Figure 2: CUDA Double `__syncthreads()` Heston (ms)

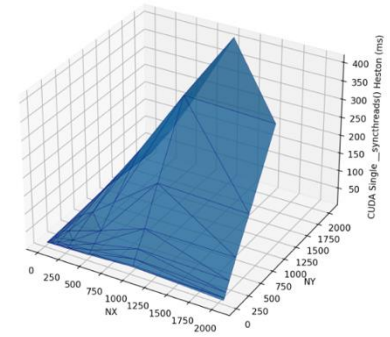


Figure 3: CUDA Single `__syncthreads()` Heston (ms)

Results

- Mean Double (ms): 53.57
- Mean Single (ms): 56.37
- Double - Single (ms): -2.80
- Instances where Double > Single: 9 / 63

Next Steps

- Implement Bates Model: Will not Attempt
 - Heston PDE + Jump Calculations = Bates PIDE
 - Integrate Integral/FFT into RHS Calculation
- Implement a non-uniform grid: Will not Attempt
- Implement more Precise ADI: Will not Attempt
- Integrate into Physics Informed Neural Network (PINN): Will not Attempt
- Test on real data: Ran out of time

References

- Ghosh, Abhijit, and Chittaranjan Mishra. "Highly efficient parallel algorithms for solving the Bates PIDE for pricing options on a GPU." *Applied Mathematics and Computation* 409 (2021): 126411.
- Heston, Steven L. "A closed-form solution for options with stochastic volatility with applications to bond and currency options." *The review of financial studies* 6, no. 2 (1993): 327-343.
- Hout, In'T., and S. Foulon. "ADI finite difference schemes for option pricing in the Heston model with correlation." *International Journal of Numerical Analysis & Modeling* 7, no. 2 (2010).
- <https://people.maths.ox.ac.uk/gilesm/cuda/>